

ATHLETE'S POV

Full-Stack Knowledge Transfer Document

A Complete A–Z Developer Reference for the AthletePOV Sports
Venue Booking Platform

USER APP

PARTNER APP

ADMIN PORTAL

Express + TypeScript · PostgreSQL + Prisma · Redis · Razorpay · Firebase FCM · React

Version 1.0 · Sprint 1–6 Complete · April 2026

■ Table of Contents

1.	Project Overview	What APOV is, its purpose, and target users
2.	System Architecture	How all components communicate
3.	Folder & File Structure	Complete backend and frontend layout
4.	Authentication System	OTP, JWT, refresh tokens, social login
5.	Venue & Slot Management	CRUD, availability, Redis slot locking
6.	Booking System	Full booking lifecycle, fee calculation, e-tickets
7.	Payment System	Razorpay integration, webhooks, fee structure
8.	Settlement & Commission	Admin-Partner payout cycle
9.	Notification System	Firebase FCM, broadcast, inbox
10	Coupon System	Creation, validation, wallet, milestones
.		
11	Tournament System	Registration, status machine, cancellation
.		
12	Content & Banners	Promotional banners for all apps
.		
13	Document & KYC System	S3 upload, admin review, partner approval
.		
14	Database Design	All 26+ tables, enums, relationships
.		
15	API Reference	107 endpoints across 13 services
.		
16	Frontend API Client	api.js, TokenStore, service objects
.		
17	Environment & Configuration	All env vars explained
.		
18	Docker & Local Setup	Step-by-step run instructions
.		
19	Security Architecture	Rate limits, RBAC, HMAC, bcrypt
.		

20	Cross-App Sync Rules	How data flows between all three apps
.		
21	Full Booking Walkthrough	End-to-end user journey
.		
22	Known Issues & Pitfalls	Bugs, gotchas, critical warnings
.		
23	How to Extend the Platform	Adding new features safely
.		

1. Project Overview

What is AthletePOV?

AthletePOV (APOV) is a multi-sided sports venue booking platform — India's answer to Hudle and KheloMore. It connects three distinct stakeholders on a single unified backend: everyday athletes who want to book a sports facility, venue owners (Partners) who list and manage their grounds, and Platform Administrators who govern the entire ecosystem. Every booking, payment, dispute, and notification flows through one Express/TypeScript API that talks to a shared PostgreSQL database.

The Three Applications

Application	Users	Primary Purpose	Key Capabilities
User App	Athletes / Players	Discover and book sports venues	Browse venues, select slots, pay online or at venue, view e-tickets, tournaments, coupons, wishlist
Partner App	Venue Owners	Manage venue listings and earnings	Create venues, configure slots, view bookings, scan e-tickets (QR), manage disputes, track analytics
Admin Portal	Platform Admins	Govern the entire platform	Approve venues/KYC, manage users & partners, set coupons, send broadcasts, view finance, manage content

Business Model & Platform Constants

The platform earns through commissions and convenience fees on every transaction. These constants are hardcoded in *bookings.service.ts* and must never be changed inline:

Constant	Value	Applies To	Description
CONVENIENCE_FEE_RATE	4%	Card / UPI bookings	Added to every online booking, charged to the user
COMMISSION_RATE	3%	All confirmed bookings	Platform takes 3% from the total booking amount
GST_RATE	18%	Platform commission	GST applied on top of the 3% commission
PAV_ONLINE_PCT	30%	Pay-at-Venue bookings	30% collected online to secure the slot
PAV_VENUE_PCT	70%	Pay-at-Venue bookings	Remaining 70% collected at the physical venue

Technology Stack

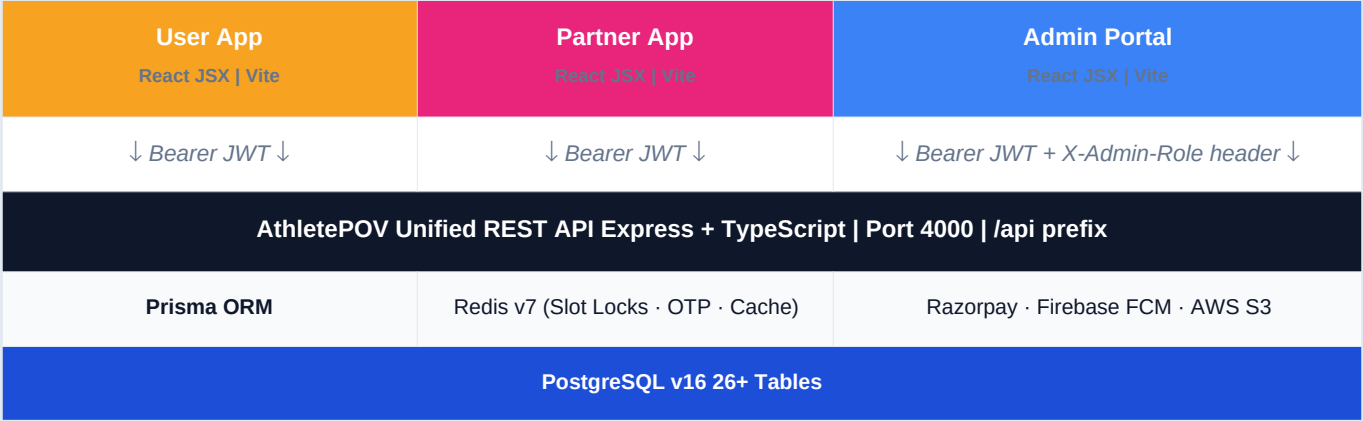
Layer	Technology	Version / Detail
Runtime	Node.js	v20+
Language	TypeScript	Strict mode
Web Framework	Express.js	v4 + Helmet, CORS, Morgan, Compression
ORM	Prisma	PostgreSQL provider

Layer	Technology	Version / Detail
Database	PostgreSQL	v16 — UUID primary keys via uuid_generate_v4()
Cache / Queue	Redis	v7 — OTP storage, slot locks, availability cache
Payments	Razorpay	Orders API, webhook HMAC verification
Push Notifications	Firebase Admin SDK (FCM)	Multi-token dispatch
File Storage	AWS S3 (or local mock)	Presigned URLs for KYC documents
Input Validation	Zod	Environment schema + request bodies
Rate Limiting	express-rate-limit	Global, auth, OTP, payment tiers
Logging	Winston	Structured JSON + HTTP request log
Frontend	React (single-file JSX)	Vite dev server, no separate CSS files
Dev Tools	VS Code + GitHub Copilot	Windows PowerShell environment

2. System Architecture

High-Level Architecture

All three frontend apps are single-page React applications. They communicate exclusively with one backend REST API. The backend follows a modular architecture: each domain (auth, venues, bookings, etc.) is isolated into its own service/controller/routes trilogy. External services — Razorpay, Firebase, S3 — are initialized once in `src/config/` and injected via singleton getters.



Request Flow

Every API request travels through this middleware chain:

1	Client sends HTTP request	Bearer token in Authorization header; X-Admin-Role for admin routes
2	Helmet	Adds security headers (XSS, HSTS, etc.)
3	CORS middleware	Validates origin against ALLOWED_ORIGINS env var
4	Raw body parser (webhook only)	express.raw() preserves Buffer for Razorpay HMAC verification
5	express.json()	Parses JSON body for all other routes (2 MB limit)
6	Request ID injection	Generates X-Request-ID UUID for tracing
7	Morgan HTTP logger	Logs method, path, status, response time to Winston stream
8	Global rate limiter	100 req / 15 min per IP (configurable)
9	Route match	Express matches to the correct module router
10	authenticate middleware	Verifies JWT, rejects expired/invalid tokens, blocks suspended accounts
11	authorize middleware	Checks role against allowed roles array
12	Controller	Parses request params, calls service function
13	Service	Business logic, Prisma queries, Redis operations
14	Error handler	Catches AppError subclasses, formats JSON error response

■ 3. Folder & File Structure

Backend Structure — src/

```
src/  
app.ts # Express app bootstrap, all middleware, route mounting  
index.ts # Entry point – starts server  
config/  
env.ts # Zod-validated environment variables (crashes on missing vars)  
prisma.ts # Prisma client singleton → import { prisma }  
redis.ts # Redis client singleton + RedisKeys helpers  
razorpay.ts # Razorpay SDK singleton + rupeesToPaise helper  
firebase.ts # Firebase Admin SDK + sendPushToToken / sendPushToMultipleTokens  
logger.ts # Winston logger (stdout + file) + httpLogStream for Morgan  
cors.ts # CORS options from ALLOWED_ORIGINS env var  
middleware/  
authenticate.ts # JWT Bearer verification → req.user = {id, role, status}  
authorize.ts # RBAC guard: authorize('user'|'partner'|'admin')  
errorHandler.ts # Global error → JSON { error: { code, message } }  
rateLimiter.ts # globalRateLimiter, authLimiter, otpLimiter, paymentLimiter  
requestId.ts # Injects X-Request-ID UUID header  
notFoundHandler.ts # 404 catch-all → AppError  
shared/  
types/index.ts # UserRole, AuthRequest, JwtAccessPayload, PaginatedQuery  
utils/  
errors.ts # AppError hierarchy (NotFound, Validation, Forbidden, etc.)  
response.ts # sendSuccess(), sendPaginated() response helpers  
modules/ # 13 domain modules – each has service + controller + routes  
auth/ # OTP request/verify, JWT refresh, logout, Google/Apple OAuth  
users/ # Profile CRUD, bookings history, wishlist toggle  
partners/ # Partner onboarding, KYC submission, profile  
venues/ # Venue CRUD, discovery, reviews, admin approval  
slots/ # Slot definitions, availability, Redis locking, demand tags  
bookings/ # Booking lifecycle, e-tickets, cancellation, disputes  
payments/ # Razorpay orders, webhook, refunds, settlements  
notifications/ # FCM dispatch, in-app inbox, admin broadcast  
coupons/ # Coupon wallet, validation, admin CRUD  
analytics/ # KPIs for user/partner/admin  
tournaments/ # Tournament listing, registration, status machine  
content/ # Banners/promotions for each app  
documents/ # KYC document upload (S3 presign), admin review
```

Frontend Structure — src/

```
src/  
api.js # Shared API client – imported by all 3 JSX apps  
AthletePOV_User_App.jsx # User App (~8,000 lines) – single-file React app  
AthletePOV_Partner_App.jsx # Partner App (~7,000 lines) – single-file React app  
AthletePOV_Admin_Portal.jsx # Admin Portal (~9,000 lines) – single-file React app
```

```
# Critical rule: ALL React components MUST be defined at file top-level,
# NEVER inside App() function – causes silent white-screen crashes

# Each file ends exactly with:
# return null;
# }
# (blank line)
# }
```

Key File Roles

File	Role	Critical Details
env.ts	Config validation	Zod schema — server crashes at startup if any required var is missing. NEVER use non-ASCII characters here.
authenticate.ts	JWT middleware	Sets req.user = {id, role, status}. Rejects blocked/suspended accounts. optionalAuthenticate for public+personalised routes.
authorize.ts	RBAC middleware	authorize('partner') userOnly partnerOnly adminOnly requireAdminHeader selfOrAdmin
bookings_service.ts	Core business logic	Contains PLATFORM constants. calculatePricing(), initiateBooking(), confirmBooking(), cancelBooking().
api.js	Frontend API client	TokenStore (localStorage per role), apiFetch with auto-refresh on 401, all service objects (Venues, Bookings, etc.)
schema.prisma	Database schema	26+ models, all using UUID PKs, Prisma Decimal for all money fields

■ 4. Authentication System

OTP-Based Login Flow

APOV uses phone-based OTP authentication as the primary login method. There is no username/password for regular users or partners. Admins use a separate flow with hardcoded credentials stored as bcrypt hashes in environment variables.

Step	Endpoint	What Happens
1. Request OTP	POST /api/auth/request-otp	Rate-limited (3 per phone per 10 min via Redis). Generates 6-digit OTP, bcrypt-hashes it, stores {hash, role} in Redis with 5-minute TTL. Simulates SMS send.
2. Verify OTP	POST /api/auth/verify-otp	Brute-force guard (max 5 attempts — auto-deletes OTP on 5th fail). bcrypt.compare(). Upserts User or Partner in DB. Issues accessToken (15m) + refreshToken (7d, stored in DB refresh_tokens table). Returns { accessToken, refreshToken, isNewAccount, role }.
3. Refresh Token	POST /api/auth/refresh	Verifies JWT signature. Validates token_id in refresh_tokens table. Rotates: revokes old token, issues new accessToken + refreshToken pair.
4. Logout	POST /api/auth/logout	Revokes refresh token in DB (sets revoked_at). Client should also clear localStorage.
5. Google OAuth	POST /api/auth/google	Verifies Google ID token, upserts oauth_identities + user record.
6. Apple OAuth	POST /api/auth/apple	Blocked in production (returns 501). Development only.

JWT Token Structure

```
// Access Token payload (JWT_ACCESS_SECRET, expires 15m)
{ sub: 'user_uuid_here', role: 'user' | 'partner' | 'admin', status: 'active' | 'blocked' | 'suspended' }

// Refresh Token payload (JWT_REFRESH_SECRET, expires 7d)
{ sub: 'user_uuid_here', role: 'user', jti: 'refresh_token_uuid_in_db' }

// Usage – all protected endpoints:
Authorization: Bearer <access_token>

// Admin endpoints additionally require:
X-Admin-Role: admin
```

Role-Based Access Control (RBAC)

Role	Who Uses It	Access Scope
user	Regular athletes booking venues	Own profile, bookings, wishlist, tournaments, coupons, notifications
partner	Venue owners	Own venues + slots, own bookings, KYC documents, analytics for own venues
admin	Platform administrators	Everything — full platform management, analytics, KYC review, finance

Security Safeguards

- OTP is **bcrypt-hashed** before Redis storage — plain OTP never persists.
- OTP is **deleted from Redis** after successful verification (single-use guarantee).
- Redis rate limit: **3 OTP requests per phone per 10 minutes**.
- **5 failed attempts** auto-deletes the OTP — prevents brute force.
- Refresh tokens are **rotated on every use** — old token is revoked immediately.
- JWT payload carries account **status** — blocked/suspended accounts are rejected at middleware layer.
- Admin endpoints require **two layers**: valid JWT with role=admin AND X-Admin-Role header.
- NEVER use non-ASCII characters (em-dashes, emoji) in env.ts or any TypeScript file — ESM loader crashes silently.

5. Venue & Slot Management

Venue Lifecycle

Status	Meaning	Who Sets It	Visible To Users?
unlisted	Newly created, awaiting admin review	System (on create)	No
listed	Approved by admin, discoverable	Admin (approve action)	Yes
suspended	Temporarily hidden by admin	Admin (suspend action)	No

Venues start as **unlisted** when created by a Partner. An Admin must approve them via `PATCH /api/venues/admin/:id/status` with `action='approve'`. Only KYC-verified partners can create venues. Featured venues are stored in a Redis sorted set (key: **featured_venues**) rather than a DB column, enabling faster reads and score-based ordering.

Slot Locking — Anti-Double-Booking

The core concurrency mechanism uses **Redis SET NX** (atomic set-if-not-exists) to guarantee that only one user can be in the payment flow for a given slot at any moment.

```
// When User A initiates booking for Slot X:
Redis SET slot:lock:X 'sessionA' NX EX 15 → 'OK' (lock acquired, proceed)

// When User B simultaneously tries Slot X:
Redis SET slot:lock:X 'sessionB' NX EX 15 → null (409 SLOT_LOCKED returned)

// After payment confirmed → slot marked 'booked' in DB
// Lock expires automatically after 15 seconds if payment doesn't complete
```

Slot Status Values

Status	Meaning	Who Sets It
available	Open for booking	Default on creation
booked	Successfully reserved by a user	Booking service on confirmation
blocked_by_partner	Partner has manually blocked this slot	Partner via <code>PATCH /:venueId/slots/:id/block</code>
blocked_by_admin	Admin has blocked this slot	Admin action

Demand Tags (Admin Pricing Control)

Admins can apply demand tags to slots to dynamically adjust pricing. When a demand tag is applied, the slot price increases by 50%. Removing the tag restores the original venue `base_price`.

Available demand tags: `peak_hours` · `high_demand` · `last_few_slots` · `flash_sale` · `premium_slot` · `weekend_special`

Redis Caching Strategy

Redis Key Pattern	TTL	What It Caches	Invalidated When
slot:lock:{slotId}	15s	Slot booking lock (NX)	Payment confirmed or expires
slots:avail:{venueId}:{date}	60s	Slot availability list for a date	Any slot write for that venue+date
featured_venues	No expiry	Sorted set of featured venue IDs	Admin pins/unpins a venue
otp:{phone}	5 min	bcrypt OTP hash + role	Verified or expired
otp:attempts:{phone}	5 min	Failed attempt counter	Reset after successful verify

■ 6. Booking System

Booking Lifecycle — Two-Step Flow

Booking is intentionally split into two steps to support Razorpay's asynchronous payment confirmation. The booking is created in PENDING state, payment happens client-side, and the webhook confirms it.

Step	Trigger	What Happens	DB State
1. Initiate	POST /api/bookings	Validates slot availability. Acquires Redis slot lock (15s). Calculates full fee breakdown. Applies coupon if provided. Checks idempotency key. Creates booking in DB.	PENDING
2. Payment	Frontend → Razorpay SDK	Client completes payment on Razorpay's hosted UI. Razorpay returns razorpay_order_id, razorpay_payment_id, signature.	PENDING
3. Confirm (Webhook)	POST /api/payments/webhook	HMAC signature verified. Razorpay order_id matched to transaction. Booking status updated. Slot marked booked. User/Partner FCM sent.	CONFIRMED
4. (Optional) Cancel	PATCH /api/bookings/:id/cancel	Booking cancelled. Slot status reset to 'available'. Refund eligible if within 24h. Redis cache invalidated. FCM notifications sent.	CANCELLED

Fee Calculation Logic

```
// CARD or UPI payment
discounted = base_amount - coupon_discount
convenience = discounted * 0.04 (4%)
total_amount = discounted + convenience
online_amount = total_amount
venue_amount = 0

// PAY AT VENUE
discounted = base_amount - coupon_discount
online_amount = discounted * 0.30 (30% - charged now)
venue_amount = discounted * 0.70 (70% - collected at venue)
total_amount = discounted
convenience = 0

// FREE SLOT
All amounts = 0, booking status immediately CONFIRMED

// Platform split (internal, not shown to user)
commission = total_amount * 0.03
gst = commission * 0.18
partner_amount = total_amount - commission - gst
```

E-Ticket System

Every booking generates a unique e-ticket code in format **APV-2026-XXXXXX** (year + 6 random hex chars). This code is stored in *bookings.eticket_code* (unique index). Partners use this code to verify player check-in via the QR scanner in the Partner App.

Dispute System

Status	Meaning
open	User has raised a dispute
under_review	Admin is reviewing
resolved	Admin has made a decision
escalated	Elevated for senior review

Resolution options:

- refund_user — full refund issued, pending refund transaction created
- warn_partner — partner warned, no refund
- closed_no_action — dispute closed without action
- escalated — elevated for further review

Idempotency Guard

To prevent double-booking from rapid re-submits (e.g., double-tap on mobile), the booking service checks Redis for an existing pending booking for the same user+slot combination before creating a new one. If found, the existing booking is returned with **already_initiated: true**. The idempotency key is: `booking:idem:{userId}:{slotId}`.

■ 7. Payment System

Razorpay Integration Flow

Step	Actor	Action
1	User App	Calls POST /api/payments/initiate with booking_id → backend creates Razorpay order, returns order_id, amount, key_id
2	Razorpay SDK (client)	Frontend opens Razorpay checkout modal, user completes payment
3	Razorpay SDK (client)	On success: returns { razorpay_order_id, razorpay_payment_id, razorpay_signature }
4	User App	Calls POST /api/payments/verify — backend verifies HMAC signature, confirms booking
5	Razorpay (async)	Sends webhook POST /api/payments/webhook — additional confirmation
6	Backend webhook handler	Verifies HMAC with RAZORPAY_WEBHOOK_SECRET, processes payment.captured / payment.failed events

Webhook Security — CRITICAL

Why Webhook Raw Body Is Critical

Razorpay signs the webhook payload with HMAC-SHA256. The signature is computed over the RAW request body bytes. If Express parses the body with `express.json()` first, the Buffer is consumed and signature verification will ALWAYS FAIL.

That's why `app.ts` has this BEFORE `express.json()`:

```
// In app.ts — ORDER IS CRITICAL
app.use('/api/payments/webhook', express.raw({ type: 'application/json' })); // FIRST
app.use(express.json({ limit: '2mb' })); // SECOND
// Verification in payments.service.ts:
const expectedSig = crypto.createHmac('sha256', env.RAZORPAY_WEBHOOK_SECRET)
  .update(rawBody) // rawBody is Buffer here
  .digest('hex');
if (expectedSig !== receivedSig) throw new ForbiddenError('Invalid webhook signature');
```

Webhook Always Returns 200

The webhook handler ALWAYS returns HTTP 200 to Razorpay, even on processing errors. This prevents Razorpay from retrying failed webhooks and creating duplicate transactions. Errors are logged internally. The response is `{ received: true, error: 'processing_failed' }` on internal errors.

Payment Endpoints

Method + Route	Auth	Purpose
POST /api/payments/initiate	user	Create Razorpay order for a pending booking
POST /api/payments/verify	user	Verify Razorpay payment signature, confirm booking
POST /api/payments/webhook	None (HMAC)	Razorpay async event handler — always 200
POST /api/payments/refund	admin	Initiate refund for a booking
GET /api/payments/booking/:id	user/admin	Get transaction records for a booking
GET /api/payments/settlements	partner	Partner settlement history
GET /api/payments/admin/transactions	admin	Full transaction ledger

■ 8. Settlement & Commission System

Weekly Settlement Cycle

Razorpay settles funds to APOV's bank account on a weekly cycle. APOV then calculates and disburses partner earnings minus platform commission. Settlement records are stored in the **settlements** table.

```
// Settlement calculation per booking:
gross_amount = booking.total_amount
platform_fee = gross_amount * 0.03 * (1 + 0.18) // 3% commission + 18% GST
net_amount = gross_amount - platform_fee

// Settlement record:
{ partner_id, period_start, period_end, gross_amount, platform_fee, net_amount, status }

// Settlement statuses: calculating → pending → on_hold → settled | failed
```

Settlement Status Flow

Status	Meaning
calculating	System is computing the settlement amount
pending	Awaiting transfer initiation
on_hold	Paused (e.g., dispute open on a booking)
settled	Funds transferred to partner's bank account
failed	Transfer failed — needs manual intervention

■ 9. Notification System

Architecture

The Notification Service is a **leaf service** — all other services call INTO it, never the reverse. Every event that triggers a notification calls `sendNotification()` which first writes to the **user_notifications** DB table (in-app inbox), then dispatches a Firebase FCM push. If FCM fails, the in-app record still persists.

FCM Token Storage

FCM tokens are stored in two places for reliability:

- **users.fcm_token** — current primary device token for a user
- **partners.fcm_token** — current primary device token for a partner
- **refresh_tokens.fcm_token** — per-device token for multi-device support

Broadcast System

Admins can send targeted broadcasts via `POST /api/notifications/admin/broadcast`. The **audience_type** determines recipients:

- `all_users`
- `all_partners`
- `specific_user`
- `specific_partner`
- `area` (geofenced)
- `sport` (sport preference)
- `all` (everyone)

Event-Triggered Notifications

Event	Recipients	Category
Booking confirmed	User + Partner	booking
Booking cancelled	User + Partner	booking
Refund initiated	User	settlement
KYC status updated	Partner	kyc
Venue approved/rejected	Partner	kyc
Admin demand tag applied	User (wishlist)	alert
New slot available	Wishlisted users	alert
Tournament cancelled	Registered users	tournament
Admin broadcast	Targeted audience	promotion/system

■ 10. Coupon System

Coupon Types & Sources

Source	Who Creates It	How
admin_created	Admin	POST /api/coupons/admin — available to all users immediately
milestone_reward	System	Auto-generated when user unlocks a milestone (first_step, hat_trick, etc.)
promo_campaign	Admin	Targeted coupon for a marketing campaign

Validation Logic

Every coupon validation checks the following in order (first failure throws):

1. Coupon exists and is_active = true
2. Current date is within valid_from and valid_until range
3. usage_limit not reached (global count)
4. order_value meets min_order_value threshold
5. User has not exceeded per_user_limit (checked via bookings.coupon_id count)

Discount Calculation

```
// PERCENT discount
discount = order_value * (discount_value / 100)
if (max_discount) discount = Math.min(discount, max_discount)

// FLAT discount
discount = Math.min(discount_value, order_value) // never exceed order value
```

Milestone System

Milestone	Trigger	Reward
first_step	User's first booking	Coupon unlocked
hat_trick	User's 3rd booking	Coupon unlocked
weekly_warrior	3 bookings in 7 days	Coupon unlocked
court_regular	10 total bookings	Coupon unlocked

■ 11. Tournament System

Tournament Lifecycle

Status	Transition	Who Can Change It
upcoming	Admin opens registration	Admin → open
open	All slots filled OR admin closes	System → full Admin → cancelled
full	Cannot register more	Auto set when registrations = max_participants
completed	After end_date passes	Admin
cancelled	Admin cancels	Admin — triggers refunds for all registrants

Registration Flow

Users register via *POST /api/tournaments/:id/register*. If *registration_fee* > 0, a Razorpay order is created for the fee. On cancellation, refunds are initiated for all paid registrations and *refunds_initiated* count is returned.

■ 12. Content & Banners

Banner System

Admins create promotional banners via *POST /api/content/admin/banners*. Banners have a **target_app** field ('user', 'partner', or 'all') that controls which apps display them. The User App homepage carousel fetches banners via *GET /api/content/banners?target=user*. Banners are active immediately upon creation (sync rule SR-5).

Field	Type	Purpose
title	String	Banner title
image_url	String	Image to display in carousel
link_url	String?	Deep-link URL when banner is tapped
target_app	String	'user' 'partner' 'all'
is_active	Boolean	Controls visibility
display_order	Integer	Sort order in carousel
valid_from / valid_until	Timestamp?	Optional time-limited campaign

■ 13. Document Verification & KYC

KYC Upload Flow

Step	Actor	Action
1. Request presigned URL	Partner	POST /api/documents/presign with { document_type, file_name, content_type } → returns upload_url (S3 presigned PUT URL) and file_url
2. Upload to S3	Partner (client)	PUT directly to S3 presigned URL — no backend involved, no bandwidth cost
3. Notify backend	Partner	Backend auto-creates PartnerDocument record with status=pending after presign (file_url stored)
4. Admin review queue	Admin	GET /api/documents/admin/pending — lists all pending KYC documents
5. Admin decision	Admin	PATCH /api/documents/:id/status with status='verified' 'rejected' + optional rejection_note
6. Auto KYC update	System	If all docs for a partner are verified, partner.kyc_status auto-updates to 'verified'

Document Types

Accepted document types: **gst_certificate** · **pan_card** · **venue_ownership** · **fssai** · **noc** · **trade_license** · **incorporation_cert** · **rental_agreement** · **cancelled_cheque** · **others**

In development mode, the presign endpoint returns a **mock URL** since AWS credentials are not configured. The AWS SDK import is dynamic, so the server starts without error even without the SDK installed.

14. Database Design

Key Design Decisions

- All primary keys are **UUID** (uuid_generate_v4()) — globally unique, not guessable
- All monetary values use **Prisma Decimal / PostgreSQL DECIMAL(10,2)** — never float
- All timestamps use **Timestamptz** (timezone-aware) — important for IST handling
- **Sport types** are stored as String[] (text array) on venues, not a join table — simpler queries
- User and Partner are **separate tables** — different onboarding, fields, and security contexts
- **Wishlist** uses a unique constraint on (user_id, venue_id) — toggle via upsert/delete pattern

All Database Tables

Table	Records	Key Fields
users	End users	user_id, phone_number, email, fcm_token, total_bookings, total_spend
partners	Venue owners	partner_id, phone_number, kyc_status, plan_tier, total_earnings, fcm_token
venues	Sports venues	venue_id, partner_id, sport_types[], status, avg_rating, base_price, slot_mode
slots	Bookable time slots	slot_id, venue_id, date, start_time, end_time, price, status, demand_tag
bookings	Booking records	booking_id, user_id, venue_id, slot_id, status, payment_mode, eticket_code
transactions	Payment records	txn_id, booking_id, razorpay_order_id, txn_type, txn_status, amount
settlements	Partner payouts	settlement_id, partner_id, gross_amount, platform_fee, net_amount, status
disputes	Booking disputes	dispute_id, booking_id, raised_by, status, resolution
refresh_tokens	JWT refresh tokens	token_id, user_id, role, token_hash, device_info, fcm_token
otp_log	OTP audit trail	otp_id, phone_number, otp_hash, expires_at, attempt_count
oauth_identities	Social login links	oauth_id, user_id, provider, provider_user_id
venue_reviews	User reviews	review_id, venue_id, user_id, booking_id, rating, comment
wishlists	User wishlisted venues	wishlist_id, user_id, venue_id
tournaments	Sports tournaments	tournament_id, venue_id, name, sport_type, status, max_participants
tournament_registrations	Team registrations	reg_id, tournament_id, user_id, team_name, payment_status
notification_broadcasts	Admin broadcasts	broadcast_id, audience_type, status, sent_at
user_notifications	In-app inbox items	notif_id, recipient_id, recipient_type, is_read

Table	Records	Key Fields
coupons	Discount codes	coupon_id, code, discount_type, discount_value, usage_limit, per_user_limit
partner_documents	KYC documents	doc_id, partner_id, document_type, file_url, status
banners	Promotional banners	banner_id, title, image_url, target_app, display_order
user_milestones	Achievement records	milestone_id, user_id, milestone_type, coupon_id

15. API Reference

Base URL: `http://localhost:4000/api`

All responses follow: `{ success: true, data: {...} }` or `{ error: { code, message, details } }`

Auth

Method	Endpoint	Auth	Description
POST	<code>/auth/request-otp</code>	Public	Request OTP for login
POST	<code>/auth/verify-otp</code>	Public	Verify OTP, get tokens
POST	<code>/auth/refresh</code>	Public	Rotate refresh token
POST	<code>/auth/logout</code>	user/partner/admin	Revoke refresh token
POST	<code>/auth/google</code>	Public	Google OAuth login
POST	<code>/auth/apple</code>	Public (dev only)	Apple OAuth login

Users

Method	Endpoint	Auth	Description
GET	<code>/users/me</code>	user	Get own profile
PATCH	<code>/users/me</code>	user	Update profile
GET	<code>/users/me/bookings</code>	user	Booking history (paginated)
GET	<code>/users/me/wishlist</code>	user	Get wishlisted venues
POST	<code>/users/me/wishlist:venueId</code>	user	Toggle wishlist

Partners

Method	Endpoint	Auth	Description
POST	<code>/partners/register</code>	partner	Complete partner onboarding
POST	<code>/partners/submit-kyc</code>	partner	Submit KYC documents
GET	<code>/partners/me</code>	partner	Full partner profile
PATCH	<code>/partners/me</code>	partner	Update partner details

Venues

Method	Endpoint	Auth	Description
GET	/venues	Public/Optional	Discover venues (filter by city, sport, price)
POST	/venues	partner	Create venue (KYC required)
GET	/venues/featured	Public	Featured venues for homepage
GET	/venues/mine	partner	Own venues list
GET	/venues/admin/all	admin	All venues with filters
GET	/venues/:id	Public/Optional	Venue detail + wishlist state
PATCH	/venues/:id	partner/admin	Update venue details
GET	/venues/:id/reviews	Public	Paginated reviews
POST	/venues/:id/reviews	user	Submit review (booking required)
PATCH	/venues/admin/:id/status	admin	approve / reject / suspend
PATCH	/venues/admin/:id/feature	admin	Pin/unpin featured

Slots

Method	Endpoint	Auth	Description
GET	/venues/:venueId/slots	Public	Slots for a specific date
GET	/venues/:venueId/slots/dates	Public	Available dates in range
POST	/venues/:venueId/slots	partner	Bulk create slots
PATCH	/venues/:venueId/slots/:id/price	partner	Update slot price
POST	/venues/:venueId/slots/:id/block	partner	Block slot
DELETE	/venues/:venueId/slots/:id/block	partner	Unblock slot
DELETE	/venues/:venueId/slots/:id	partner	Delete slot
POST	/slots/admin/demand-tag	admin	Apply demand tag (+50% price)
DELETE	/slots/admin/demand-tag	admin	Remove demand tag

Bookings

Method	Endpoint	Auth	Description
GET	/bookings/preview	Public	Preview fee breakdown before booking
POST	/bookings	user	Initiate booking (step 1)
GET	/bookings	user	Own bookings list
GET	/bookings/:id	user/partner/admin	Booking detail
PATCH	/bookings/:id/cancel	user/admin	Cancel booking

Method	Endpoint	Auth	Description
GET	/bookings/:id/ticket	user/partner	Get e-ticket
POST	/bookings/:id/dispute	user	Raise dispute
GET	/bookings/admin/all	admin	All bookings with filters
POST	/bookings/:id/checkin	partner	QR check-in verification
PATCH	/disputes/:id/resolve	admin	Resolve dispute

Payments

Method	Endpoint	Auth	Description
POST	/payments/initiate	user	Create Razorpay order
POST	/payments/verify	user	Verify payment signature
POST	/payments/webhook	None/HMAC	Razorpay webhook handler
POST	/payments/refund	admin	Initiate refund
GET	/payments/booking/:id	user/admin	Transaction records for booking
GET	/payments/settlements	partner	Settlement history
GET	/payments/admin/transactions	admin	Full transaction ledger

Notifications

Method	Endpoint	Auth	Description
GET	/notifications	user/partner	Inbox with unread count
PATCH	/notifications/:id/read	user/partner	Mark single as read
PATCH	/notifications/read-all	user/partner	Mark all as read
POST	/notifications/admin/broadcast	admin	Send broadcast
GET	/notifications/admin/broadcasts	admin	Broadcast history

Coupons

Method	Endpoint	Auth	Description
GET	/coupons	user	User's coupon wallet
GET	/coupons/validate	user	Validate code + get discount
GET	/coupons/:code	user	Get coupon by code
GET	/coupons/admin	admin	All coupons
POST	/coupons/admin	admin	Create coupon

Method	Endpoint	Auth	Description
PATCH	/coupons/admin/:id	admin	Update coupon
DELETE	/coupons/admin/:id	admin	Delete coupon

Analytics

Method	Endpoint	Auth	Description
GET	/analytics/user/me	user	Personal stats + milestones
GET	/analytics/partner/:id/dashboard	partner	Partner KPIs
GET	/analytics/partner/:id/revenue	partner	Monthly revenue trend
GET	/analytics/admin/dashboard	admin	Platform snapshot
GET	/analytics/admin/trends	admin	Daily/weekly trends
GET	/analytics/admin/top-venues	admin	Top venues by bookings

Tournaments

Method	Endpoint	Auth	Description
GET	/tournaments	Public	Browse tournaments
POST	/tournaments	partner	Create tournament
GET	/tournaments/:id	Public	Tournament detail
PATCH	/tournaments/:id	partner/admin	Update tournament
PATCH	/tournaments/:id/status	admin	Change tournament status
POST	/tournaments/:id/register	user	Register for tournament
GET	/tournaments/:id/registrations	partner	View registrations
GET	/tournaments/me	user	My tournament registrations

Content

Method	Endpoint	Auth	Description
GET	/content/banners	Public	Get banners for target app
GET	/content/admin/banners	admin	All banners
POST	/content/admin/banners	admin	Create banner
PATCH	/content/admin/banners/:id	admin	Update banner
DELETE	/content/admin/banners/:id	admin	Delete banner
PATCH	/content/admin/banners/order	admin	Reorder banners

Documents

Method	Endpoint	Auth	Description
POST	/documents/presign	partner	Get S3 presigned upload URL
GET	/documents	partner	Own documents list
GET	/documents/admin/pending	admin	Pending KYC queue
PATCH	/documents/:id/status	admin	Verify or reject document

■ 16. Frontend API Client (api.js)

Overview

api.js is the single shared API client used by all three React apps. It handles auth token storage, automatic token refresh on 401, and provides typed service objects for every backend module. Import it at the top of each JSX file.

```
import { Auth, Venues, Slots, Bookings, Payments, Notifications,
Coupons, Analytics, Tournaments, Content, Documents,
Disputes, Partners, Users, TokenStore,
openRazorpayCheckout } from './api.js';
```

TokenStore — Per-Role Token Management

Tokens are stored in localStorage with role-specific keys:

```
localStorage key Value
apov_token_user User JWT access token
apov_refresh_user User JWT refresh token
apov_user_user User object (JSON)
apov_token_partner Partner JWT access token
apov_token_admin Admin JWT access token

// Usage
TokenStore.get('user') // get access token
TokenStore.set('user', token) // store access token
TokenStore.clear('user') // clear all keys for role
TokenStore.getUser('user') // get stored user object
```

Auto Token Refresh on 401

When any API call returns 401 (token expired), apiFetch automatically calls *POST /auth/refresh* with the stored refresh token, updates localStorage, and retries the original request. Multiple concurrent 401 responses are queued and resolved with a single refresh call (no race condition).

Razorpay Checkout Helper

```
// Opens Razorpay modal, handles payment, verifies with backend
await openRazorpayCheckout({
  bookingId: 'booking_uuid',
  amount: 83200, // paise (rupees * 100)
  currency: 'INR',
  keyId: 'rzp_test_...',
  onSuccess: (result) => { /* booking confirmed */ },
  onFailure: (err) => { /* payment failed */ },
});
```

17. Environment & Configuration

Required Environment Variables (.env)

Variable	Required	Example / Default	Notes
DATABASE_URL	Yes	postgresql://user:pass@localhost:5432/apov_db	Prisma connection string
JWT_ACCESS_SECRET	Yes	random 32+ char string	Signs access tokens (15m)
JWT_REFRESH_SECRET	Yes	random 32+ char string	Signs refresh tokens (7d)
JWT_ACCESS_EXPIRES_IN	No	15m	
JWT_REFRESH_EXPIRES_IN	No	7d	
REDIS_URL	Yes	redis://localhost:6379	Container: athletes-redis
RAZORPAY_KEY_ID	Yes	rzp_test_...	From Razorpay dashboard
RAZORPAY_KEY_SECRET	Yes	...	From Razorpay dashboard
RAZORPAY_WEBHOOK_SECRET	Yes	...	Set in Razorpay webhook settings
FIREBASE_PROJECT_ID	Yes	apov-prod	Firebase console
FIREBASE_PRIVATE_KEY	Yes	-----BEGIN-----	Service account JSON field
FIREBASE_CLIENT_EMAIL	Yes	...@...iam.gserviceaccount.com	Service account email
FIREBASE_DATABASE_URL	Yes	https://...firebaseio.com	
AWS_ACCESS_KEY_ID	Optional	AKIA...	Required for S3 uploads
AWS_SECRET_ACCESS_KEY	Optional	...	Required for S3 uploads
AWS_S3_BUCKET	Optional	apov-kyc-docs	Required for S3 uploads
AWS_REGION	No	ap-south-1	Mumbai region
ALLOWED_ORIGINS	No	http://localhost:5173	CORS whitelist (comma-separated)
PORT	No	4000	API server port
LOG_LEVEL	No	info	error/warn/info/http/debug
RATE_LIMIT_MAX_REQUESTS	No	100	Per IP per 15 min window

CRITICAL — UTF-8 Warning

NEVER allow non-ASCII characters (em-dashes —, smart quotes ", emoji) in env.ts or ANY TypeScript source file. The ESM loader crashes silently with [Object: null prototype] error. This has caused a production incident. Use only standard ASCII in all TypeScript comments and strings.

18. Docker & Local Setup

Docker Compose Services

Service	Container Name	Port	Image
PostgreSQL	apov_postgres	5432	postgres:16-alpine
Redis	apov_redis	6379	redis:7-alpine (password: redispassword)
API	apov_api	4000	Built from local Dockerfile

Step-by-Step Local Setup

Install dependencies

```
npm install
```

Configure environment

```
cp .env.example .env
# Fill in DATABASE_URL, JWT secrets, Razorpay keys, Firebase config
```

Start Docker services

```
docker-compose up -d postgres redis
```

Run database migrations

```
npx prisma migrate dev --name init
npx prisma generate
```

Start API server

```
npm run dev
# Server starts at http://localhost:4000
# Health check: GET /health
```

Start frontend (separate terminal)

```
cd ../athletespov-frontend
npm install
npm run dev
# Vite server at http://localhost:5173
```

Verify

```
curl http://localhost:4000/health
# Should return: { status: 'ok', service: 'Athlete's POV API' }
```

TypeScript Compile Check

```
npx tsc --noEmit # Must show zero errors before running
```

```
# Common error: 'Cannot find module' → check import paths  
# Common error: 'Property does not exist' → run npx prisma generate  
# Port 3000/4000 in use → kill existing node: taskkill /F /IM node.exe (Windows)
```

■ ■ 19. Security Architecture

Rate Limiting Tiers

Limiter	Applied To	Limit
globalRateLimiter	All routes	100 requests / 15 min per IP
authLimiter	/auth routes	10 requests / 15 min per IP
otpLimiter	/auth/request-otp	3 requests / 10 min per IP
paymentLimiter	/payments routes	20 requests / 1 min per IP
verifyOtpLimiter	/auth/verify-otp	5 attempts then block
refreshLimiter	/auth/refresh	10 requests / 15 min per IP

Security Checklist

- JWT access tokens expire in **15 minutes** — short window limits exposure
- Refresh token rotation on every use — compromised tokens invalidated immediately
- OTP bcrypt-hashed before Redis storage — plain OTP never persists anywhere
- Razorpay webhook HMAC-SHA256 verification — prevents fake webhook injection
- Helmet.js sets security headers — XSS protection, HSTS, CSP, etc.
- CORS whitelist via ALLOWED_ORIGINS — prevents unauthorized cross-origin requests
- Admin double-auth: valid JWT role=admin + X-Admin-Role header required
- Zod environment schema — server refuses to start with invalid/missing config
- No hardcoded credentials in source code — all sensitive values via env vars
- Brute-force protection on OTP — auto-deletes after 5 failed attempts
- Redis slot lock (NX) — prevents race conditions on concurrent bookings
- Account status check in JWT middleware — blocked users rejected at every request

■ 20. Cross-App Synchronization Rules

These 18 sync rules define how data and events propagate across all three apps. All are satisfied in Sprint 6 via the backend API + frontend integration.

Rule	Trigger	Effect
SR-1	Booking created	Partner dashboard shows new booking; Admin booking count increments
SR-2	Partner blocks slot	Users see 'Sold Out' immediately on slot grid
SR-3	Admin applies demand tag	Price updates reflected in User App; Optional FCM push
SR-4	Admin creates coupon	Appears in User App coupon wallet immediately
SR-5	Admin publishes banner	User App homepage carousel updates immediately
SR-6	Admin approves partner	FCM notification sent to partner; Status updates in Partner App
SR-7	Partner adds venue	Visible in User App after admin approval
SR-8	Admin cancels booking	FCM to user and partner; Refund initiated; Slot reopens
SR-9	Admin reassigns slot	E-ticket updates with new slot info; FCM notifications sent
SR-10	Partner records offline booking	Appears in Admin Finance section
SR-11	Admin sends push notification	Appears in both User App and Partner App inboxes
SR-12	User wishlists a venue	User receives push when new slots open for that venue
SR-13	Partner uploads KYC doc	Appears in Admin pending documents queue immediately
SR-14	User registers for tournament	Partner sees registration count decrement; Admin notified
SR-15	Admin toggles homepage layout	User App reflects changes on next load
SR-16	User selects PAV payment	Partner booking card shows PAV badge; Admin Finance tracks it
SR-17	User cancels booking	Slot immediately reopens; FCM to user and partner; Refund eligible
SR-18	Admin changes commission rate	Applied from next settlement cycle — historical settlements unaffected

■ 21. Complete Booking Flow Walkthrough

End-to-End: User Books a Football Slot

1	App Launch	User App loads. Checks localStorage for apov_token_user. If found and not expired, navigates to home screen.
2	OTP Login	User enters phone. POST /auth/request-otp. OTP sent (simulated). User enters OTP. POST /auth/verify-otp. JWT access + refresh tokens stored in localStorage.
3	Browse Venues	User taps 'Football'. GET /venues?sport=football&city=Mumbai. Returns array of listed venues with is_wishlisted flag if user is logged in.
4	View Venue	User taps a venue. GET /venues/:id returns full venue details, photos, amenities, avg_rating.
5	Select Date	User picks a date on the calendar. GET /venues/:venueId/slots/dates fetches available dates (green dots).
6	Select Slot	User picks a time slot. GET /venues/:venueId/slots?date=2026-05-01 returns slot list with price and status.
7	Preview Fees	GET /bookings/preview?slot_id=...&payment_mode=card returns fee breakdown: base, convenience_fee, total.
8	Apply Coupon (optional)	GET /coupons/validate?code=SAVE20&_value=800 returns discount_amount=160.
9	Initiate Booking	POST /bookings with { venue_id, slot_id, payment_mode: 'card', coupon_code: 'SAVE20' }. Backend: acquires Redis slot lock (15s), creates PENDING booking, returns booking_id + fees.
10	Create Razorpay Order	POST /payments/initiate with { booking_id }. Backend creates Razorpay order. Returns { razorpay_order_id, amount, key_id }.
11	Open Razorpay Modal	Frontend opens Razorpay checkout. User completes UPI/card payment.
12	Verify Payment	POST /payments/verify with { razorpay_order_id, razorpay_payment_id, razorpay_signature }. Backend verifies HMAC. Booking confirmed. Slot marked 'booked'. FCM sent to user + partner.
13	Webhook (async)	Razorpay sends POST /payments/webhook. Backend verifies HMAC again. Idempotency check prevents double-processing.
14	View E-Ticket	GET /bookings/:id/ticket returns eticket_code APV-2026-XXXXXX, venue, slot, payment breakdown.
15	Partner Check-in	Partner opens Partner App, taps QR Scanner. Scans user's e-ticket QR. POST /bookings/:id/checkin with e-ticket token. Returns booking detail.
16	Post-Play Review	After booking completed, user submits review. POST /venues/:id/reviews with rating + comment.
17	Milestone Check	System checks if user has reached hat_trick (3rd booking) or other milestones. If yes, coupon automatically generated.
18	Settlement (weekly)	Admin sees booking in analytics. At end of settlement period, partner receives NET amount (gross - 3% commission - 18% GST on commission) via Razorpay payout.

22. Known Issues & Critical Pitfalls

CRITICAL	Non-ASCII in TypeScript files Copilot sometimes inserts em-dashes (—) and emoji in comments. This corrupts UTF-8 and causes the ESM loader to crash with [Object: null prototype] error — no stack trace. Fix: scan all .ts files with grep for non-ASCII chars.
CRITICAL	Component defined inside App() Defining React components inside the App() function causes white-screen crashes or constant remounting with no console errors. ALL components must be at file top-level.
CRITICAL	Webhook raw body order in app.ts app.use('/api/payments/webhook', express.raw(...)) MUST appear before app.use(express.json(...)). Reversing this breaks HMAC verification for all Razorpay webhooks.
HIGH	EADDRINUSE on port 3000/4000 Kill existing Node process before restarting: taskkill /F /IM node.exe (Windows) or kill -9 \$(lsof -ti:4000) (Mac/Linux).
HIGH	Redis container name The actual running container is named 'athletes-redis', not 'redis'. Use this exact name for connectivity checks.
HIGH	Prisma Decimal type Prisma returns Decimal objects for money fields (price, base_price, etc.). Math operations require Number() cast: Number(slot.price) * 1.5. JSON serialization is fine without cast.
HIGH	f24() time formatter The frontend time formatter breaks when the backend returns Time database objects instead of plain strings. Always unwrap/format times before passing to f24().
MEDIUM	TokenStore.isLoggedIn does not exist Use !!TokenStore.get('user') instead. auth.isLoggedIn is a different path.
MEDIUM	Users.updateLocation does not exist Use Users.update() with location data in the body.
MEDIUM	Slot display is always empty venue.slots is always empty for real API venues. Fetch slots separately via GET /venues/:id/slots?date=...
MEDIUM	Admin venue creation POST /venues is partner-only. Use dedicated POST /venues/admin/create for admin-created venues.
MEDIUM	Extra closing braces from Copilot After every Copilot-applied backend change, verify brace balance. Copilot frequently adds extra } at file end.
LOW	Antigravity stale cache Antigravity reads files at session start. After any file change, start a new Antigravity session to get fresh content.
LOW	PowerShell download path Downloads go to C:\Users\Admin\Downloads\ — NOT C:\Downloads\

INFO

Apple OAuth blocked in production

Returns 501. Apple login is dev-only. Do not expose to production users.

■ 23. How to Modify & Extend the Platform

Adding a New Backend Module

1. Create folder: `src/modules/your_module/`
2. Create `your_module.service.ts` — business logic, Prisma queries
3. Create `your_module.controller.ts` — HTTP parsing, calls service functions
4. Create `your_module.routes.ts` — Express router with `authenticate` + `authorize` middleware
5. Import and mount in `app.ts`: `app.use(`${API}/your_module`, yourModuleRoutes);`
6. Add Zod validation schemas if needed
7. Run `npx tsc --noEmit` — zero errors required before running
8. Test every endpoint with curl or Postman before connecting frontend

Adding a New Frontend Feature

- **NEVER rewrite entire JSX files** — always use targeted surgical edits (`str_replace` pattern)
- Define ALL new components at **file top-level**, never inside `App()`
- Add new state variables inside the existing `App()` `useState` declarations block
- Wire API calls via the service objects in `api.js` — do not call `fetch()` directly
- If API fails, fall back to existing mock data (additive pattern)
- Verify JSX file ends exactly with: `return null; → } → blank line → }`
- Check browser console for errors after every change

Adding a New Database Table

1. Add the model to `schema.prisma` following existing patterns (UUID PK, `Timestamptz` for dates)
2. Run: `npx prisma migrate dev --name add_your_table`
3. Run: `npx prisma generate` (updates TypeScript types)
4. Run: `npx tsc --noEmit` to verify no type errors
5. Import the new model in your service via `{ prisma }` from `'../config/prisma'`

Feature Batch Workflow (Recommended)

Following the established sprint pattern, new features should be delivered as:

- **Backend Copilot prompts** — structured numbered prompts for each service/controller change
- **Frontend JSX patch file** — downloadable `_Latest.jsx` with PowerShell copy commands
- **Verification checklist** — `npx tsc --noEmit` passes, each endpoint tested before frontend applied

Three-Tool Division of Labor

Tool	Used For
Claude Project Chat	Cross-app analysis, architecture decisions, downloadable patch files, knowledge transfer
VS Code + GitHub Copilot	Targeted backend TypeScript changes, applying structured Copilot prompts
Antigravity	Live codebase reading, verification after changes, iterative debugging

Platform Summary

- 13 Backend Modules · 107+ API Endpoints · 6 Sprints Complete
- 26+ Database Tables · 60+ Enum Values · UUID PKs throughout
- 3 Frontend Apps · ~24,000 lines of React JSX total
- 18 Cross-App Sync Rules · 25 Gap Closures (Sprint 6)
- Razorpay (payments) · Firebase FCM (notifications) · AWS S3 (KYC docs)
- Redis slot locking · JWT rotation · bcrypt OTP · Zod env validation

This document was generated from the live project source files. Always verify against the actual codebase for the most current implementation details.